



PolyMarket: V2 Diff review

Security Review

Cantina Managed review by:

Giovanni Di Siena, Lead Security Researcher

Red Swan, Security Researcher

June 3, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	6
3.1	Low Risk	6
3.1.1	Missing Validation Between Prepared Condition and Taker Order	6
3.1.2	Insecure Public Migration of Legacy CTF Positions	6
3.1.3	Combinatorial positions cannot be redeemed if losing legacy subconditions have not been fully reported	7
3.1.4	Round-tripped spoke results can bypass migrated neg-risk settlement on the hub chain	8
3.2	Informational	9
3.2.1	No-Op Constructor Call in <code>PositionManager</code>	9
3.2.2	Redundant Event Emission Without State Transitions	9
3.2.3	Recomposition can increase combinatorial payouts compared to the sum of rounded fractional payouts	10
3.2.4	Potential ABI Mismatch Between <code>CollateralToken</code> and Ramps	15

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From May 18th to May 22nd the Cantina team conducted a review of [polymarket-v2](#) on commit hash [8009ad91](#). The team identified a total of **8** issues:

Issues Found

Severity	Count	Fixed	Partially Fixed	Acknowledged
Critical Risk	0	0	0	0
High Risk	0	0	0	0
Medium Risk	0	0	0	0
Low Risk	4	2	1	1
Gas Optimizations	0	0	0	0
Informational	4	1	0	3
Total	8	3	1	4

2.1 Scope

The security review had the following components in scope for [polymarket-v2](#) on commit hash [8009ad91](#):

```
src
├── abstract
│   └── ERC1155TokenReceiver.sol
├── auth
│   ├── InitializableRoles.sol
│   └── Roles.sol
├── bridge
│   ├── CcipBridge.sol
│   ├── abstract
│   │   └── BridgeBase.sol
│   └── interfaces
│       ├── IBridge.sol
│       └── IBridgeCallback.sol
├── collateral
│   ├── CollateralOfframp.sol
│   ├── CollateralOnramp.sol
│   ├── CollateralToken.sol
│   ├── PermissionedRamp.sol
│   ├── abstract
│   │   ├── CollateralErrors.sol
│   │   └── Pausable.sol
│   ├── dev
│   │   ├── CollateralSetup.sol
│   │   └── CollateralSetup.t.sol
│   └── interfaces
│       ├── ICollateralToken.sol
│       └── ICollateralTokenCallbacks.sol
├── dev
│   ├── Addresses.sol
│   ├── DeployLib.sol
│   ├── TestHelper.sol
│   └── Vm.sol
├── exchange
│   ├── Exchange.sol
│   ├── MATCH_ORDERS_LIFECYCLE.md
│   └── OrderStructs.sol
├── external
│   └── uma
│       └── interfaces
│           └── IAddressWhitelist.sol
```

- └─ IFinder.sol
- legacy
 - └─ dev
 - └─ NegRiskAdapterSetUp.sol
 - └─ interfaces
 - └─ IConditionalTokens.sol
 - └─ IConditionalTokensMethods.sol
 - └─ INegRiskAdapter.sol
 - └─ IUmaCtfAdapter.sol
 - └─ libraries
 - └─ CTFHelpers.sol
 - └─ CTHelpers.sol
 - └─ NegRiskIdLib.sol
- libraries
 - └─ BridgePayloads.sol
 - └─ CrossChainTypes.sol
 - └─ Ids.sol
 - └─ ModuleIds.sol
- modules
 - └─ BinaryModule.sol
 - └─ CombinatorialModule.sol
 - └─ NegRiskModule.sol
 - └─ abstract
 - └─ BaseModule.sol
 - └─ ModuleErrors.sol
 - └─ OracleModule.sol
 - └─ dev
 - └─ ModuleProxyLib.sol
 - └─ interfaces
 - └─ ICombinatorialModuleCallback.sol
 - └─ IModuleCallback.sol
 - └─ INegRiskModuleCallbacks.sol
 - └─ migration
 - └─ BaseMigrationMixin.sol
 - └─ BinaryMigrationMixin.sol
 - └─ NegRiskMigrationMixin.sol
- oracle
 - └─ GLOSSARY.md
 - └─ OracleAggregator.sol
 - └─ README.md
 - └─ abstract
 - └─ OracleAggregatorErrors.sol
 - └─ OracleAggregatorEvents.sol
 - └─ OracleModuleBase.sol
 - └─ interfaces
 - └─ IArbitratorModule.sol
 - └─ IBinaryReporter.sol
 - └─ IDisputerModule.sol
 - └─ IOptimisticOracleV2.sol
 - └─ IOptimisticRequester.sol
 - └─ IOracleAggregator.sol
 - └─ IReporterModule.sol
 - └─ libraries
 - └─ OptimisticOraclePayoutLib.sol
 - └─ mixins
 - └─ Auth.sol
 - └─ Pausable.sol
 - └─ modules
 - └─ OptimisticOracleModule.sol
 - └─ README.md
 - └─ reporters
 - └─ ChainlinkReporterModule.sol
 - └─ EOAReporterModule.sol

```
|— positionManager
|   |— IPositionManagerModule.sol
|   |— PositionManager.sol
|   |— dev
|       |— PositionManagerSetup.sol
|— routers
|   |— BridgeRouter.sol
|   |— CtfRouter.sol
|   |— Router.sol
|   |— dev
|       |— RouterSetup.sol
|— utils
|   |— AutoRedeemer.sol
```

3 Findings

3.1 Low Risk

3.1.1 Missing Validation Between Prepared Condition and Taker Order

Severity: Low Risk

Context: [Exchange.sol#L562-L572](#)

Description: The `Exchange.matchOrdersAndPrepareCombinatorial()` function prepares a supplied combinatorial leg array and then matches orders, but it does not verify that the prepared combinatorial condition is the same condition represented by the taker order's `tokenId`.

The `ConditionId` returned by `prepareCondition()` is ignored. As a result, the function can prepare one combinatorial condition while matching orders for a different condition. While the Exchange operator is trusted to submit correct payloads, this missing check creates a risk of operational errors where a parlay fill is associated with a different basket than the one prepared in the same transaction.

If the leg payload and the taker order's condition ID diverge, holders of the resulting combinatorial position may be unable to use settlement and restructuring paths that require the leg definition to be correctly stored (e.g., `redeem`, `compress`, `unwrap`, `extract`, `inject`, or `convert`). This impacts user-level correctness and the recoverability of positions.

Recommendation: Bind the prepared condition to the taker order by validating the returned `ConditionId` against the `takerOrder.tokenId`:

```
ConditionId conditionId =
    ↳ CombinatorialModule(COMBINATORIAL_MODULE).prepareCondition(combinatorialLegs);
if (takerOrder.tokenId.conditionId() != conditionId) {
    revert InvalidCombinatorialCondition();
}
```

Consider also requiring that `takerOrder.tokenId.moduleId()` is the combinatorial module for this entry point to ensure consistency.

Polymarket: Fixed in commit [935446f](#).

Cantina Managed: Verified. The prepared combinatorial condition identifier is now compared against that specified by the taker order.

3.1.2 Insecure Public Migration of Legacy CTF Positions

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The `BaseMigrationMixin.migratePositions()` function allows users and operators to publicly migrate legacy CTF positions into V2 positions. This introduces hybrid legacy/V2 inventory states that significantly expand the protocol's attack surface and have already exposed violations of settlement and accounting invariants across migration resolution, `NegRisk` conversion, and vault accounting.

Publicly reachable migration creates several dangerous scenarios:

1. **Vault Accounting Mismatch:** Users can migrate legacy `NegRisk` loser positions and later receive V2 collateral through `convert()` and redemption before the corresponding legacy inventory has been settled into the V2 vault. The backing may still exist as a legacy claim, but the V2 vault is uncredited at payout time.
2. **Legacy Settlement Bypass:** Migration-backed conditions can be finalized through native V2 reporting paths instead of `resolveMigrationCondition()`. Once a native result is stored, migration settlement can be blocked, stranding module-held legacy inventory that should have credited the V2 vault.

3. **Bypassing Legacy Fee Rules:** Migrated NegRisk markets do not preserve legacy fee structures. Public V2 operations can recreate legacy conversion payoffs without enforcing the original fee haircut or routing the fee tranche to the intended receiver.

Supporting arbitrary, user-driven migration is materially more complex than the core product requirement of "V2 parlays reading legacy results" and introduces unnecessary risk to V2 vault accounting.

Recommendation: Disable `migratePositions()` overloads in `BaseMigrationMixin.sol`. This can be done simply by making a revert the first operation and can later be removed and upgraded away.

Polymarket: Acknowledged. Fixed migrated resolutions in [PR 282](#).

Cantina Managed: Partially fixed.

3.1.3 Combinatorial positions cannot be redeemed if losing legacy subconditions have not been fully reported

Severity: Low Risk

Context: `BaseModule.sol#L147-L149`, `CombinatorialModule.sol#L1335`, `CombinatorialModule.sol#L1342-L1346`, `BaseMigrationMixin.sol#L150-L154`, `NegRiskModule.sol#L162-L166`

Description: For non-migrated neg-risk events, `resolveConditionToNo()` can infer losing conditions once `resultsSum[eventId] == RESULT_DENOMINATOR` and store `result[loserCondition] = [0, RESULT_DENOMINATOR]`. Migrated neg-risk events, however, execute through `_resolveMigrationCondition()` which requires legacy CTF payout data to be present. This is enforced by requiring the payout denominator as the sum of numerators to be greater than zero. If `reportOutcome(questionId, false)` has not been called for a legacy neg-risk losing condition, then those numerators are still `[0, 0]`, so any attempt at calling `resolveMigrationCondition()` reverts with `ConditionNotResolved()`.

Therefore, for a combinatorial position `YES(A) ^ NO(B)` in which A and B are subconditions of the same legacy neg-risk event, redemption will be blocked despite `reportOutcome(legacyA, true)` being called. Once A has reported true, B is economically known to be `NO`, so the parlay should be redeemable. However, assuming `legacyB` has not explicitly been reported false then `resolveConditionToNo(B)` cannot infer `NO(B)` pays out, and because subcondition B has no stored result then `NO(B)` appears unresolved and the call reverts with `PositionNotRedeemable()`.

Since `NegRiskAdapter` does not have a short-circuit to resolve all other conditions to `NO` once a single condition resolves to `YES`, this means that outcomes need to be reported for every subcondition of legacy neg-risk event, regardless of the payout. Despite having a logical branch, `resolveConditionToNo()` does not work correctly for migrated neg-risk markets unless all legacy conditions have been fully reported.

Recommendation: Ensure that all legacy payouts, including the winner and all losing conditions, are reported. Do not remove the migration-aware short-circuit within `resolveConditionToNo()` since this function is callable once all conditions are reported and the migration condition branch is needed to ensure legacy positions are redeemed and collateral is settled to the vault.

Polymarket: Acknowledged. Operationally we make sure to call `reportOutcome(questionId, false)` on the legacy UMA oracle for every losing subcondition of a migrated neg-risk event, not just the winner. The constraint is documented in NatSpec (`NegRiskModule.sol:165-166`) and tested in `NegRiskMigration.t.sol:775`.

Cantina Managed: Acknowledged.

3.1.4 Round-tripped spoke results can bypass migrated neg-risk settlement on the hub chain

Severity: Low Risk

Context: BridgeBase.sol#L485-L486, NegRiskMigrationMixin.sol#L97-L98, NegRiskModule.sol#L121-L131, NegRiskModule.sol#L162-L168

Description: Once all legacy CTF conditions have been reported, calls to `NegRiskModule.resolveConditionToNo()` route migrated losers through `_resolveMigrationCondition()`. This ensures that the module's legacy CTF position is redeemed and settled into the vault before V2 `NO` holders can realize the collateral backing. `BridgeBase._handleResultReceive()`, however, is not migration-aware and instead forwards incoming results to `NegRiskModule.reportResult()` which calls `_finalizeNegriskResolution()` directly, even for migrated conditions.

If a winning `YES` result is permissionlessly bridged to a spoke chain, `resolveConditionToNo()` can be called for every remaining loser, even if the legacy CTF payouts have not yet been reported. This is because spoke `NegRiskModule` deployments do not set legacy addresses and so those condition IDs are never prepared as migration conditions. These loser results can then be bridged back to the hub chain through calls to `reportResult()`, marking them as resolved to `NO` without ever redeeming the corresponding legacy position or settling legacy collateral.

Any subsequent attempts to resolve losing migration conditions on the hub chain revert with `ConditionAlreadyResolved()`, meaning redemption via `_redeemLegacyPositions()` and settlement via `_settleLegacyCollateralToVault()` can never proceed. Migrated loser `NO` positions can therefore redeem collateral on the hub chain against an unresolved and, in the absence of a contract upgrade, unredeemable legacy backing position.

Proof of Concept: Consider the following scenario:

1. A neg-risk migration event exists on the hub chain. An unmatched loser legacy `NO` position is migrated, meaning the module now holds the legacy `NO` token and must route through `_resolveMigrationCondition()` to redeem and settle to the vault.
2. The legacy CTF result is reported for the winning condition and relayed to the V2 module on the hub chain. Resolution executes through `resolveMigrationCondition()`, which sets `resultsSum[eventId] = 1_000_000` for the event.
3. An arbitrary caller permissionlessly invokes `bridgeResult()` to bridge that winner result to a spoke chain.
4. The bridged winner makes `resultsSum[eventId] == 1_000_000` on the spoke chain, meaning any caller can execute `resolveConditionToNo()` on the spoke chain for each unresolved loser, storing `[0, 1_000_000]` payouts in the `result` mapping.
5. Assuming `bridgeResult()` is also enabled in the opposite direction, from the spoke chain to the hub chain, any caller can now bridge any number of these spoke chain loser results back to the hub chain.
6. `_handleResultReceive()` invokes `reportResult()` rather than `_resolveMigrationCondition()`, so the loser `NO` result is stored through `_finalizeNegriskResolution()` without ever reading legacy payouts, redeeming the module's legacy CTF position, or settling wcol/USDC.e into the vault.
7. Given the `result` mapping is now populated for the migrated `NO` position, it can be redeemed for pUSD via `BaseModule.redeem()`.

Recommendation: Consider adding an admin/operator function to forcibly redeem and settle module-held loser positions.

Polymarket: Fixed in commits [140647f](#) and [fb38d27](#).

Cantina Managed: Verified. Bridged unresolved migration conditions now first report through the `_resolveMigrationCondition()` and verify that the result being proposed by the bridge is equal to that from the CTF.

3.2 Informational

3.2.1 No-Op Constructor Call in `PositionManager`

Severity: Informational

Context: `PositionManager.sol`#L105-L109

Description: The `PositionManager` constructor explicitly calls `InitializableRoles()` even though the `InitializableRoles` contract does not define a constructor.

The call is a no-op. While it does not introduce an initialization bug—since `PositionManager.initialize()` correctly handles ownership and role assignment—the explicit call is superfluous. It may lead reviewers to believe that `InitializableRoles` performs constructor-time setup, which is not the case for this proxy-compatible base contract.

Recommendation: Remove the redundant `InitializableRoles()` call from the `PositionManager` constructor to improve code clarity and maintainability.

Polymarket: Fixed in commit `d3a51a7`.

Cantina Managed: Verified.

3.2.2 Redundant Event Emission Without State Transitions

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: Several setter-style functions in the protocol emit state-change events even when the requested value is already the current value. This allows privileged callers, operators, or users to produce duplicate logs (e.g., pause, unpause, preapproval) that do not correspond to an actual state transition.

For example, in `Exchange.sol`:

```
function pauseTrading() external onlyRoles(ADMIN_ROLE) {
    paused = true;
    emit TradingPaused(msg.sender);
}

function invalidatePreapprovedOrder(bytes32 orderHash) external onlyOperator {
    preapproved[orderHash] = false;
    emit OrderPreapprovalInvalidated(orderHash);
}
```

Other instances include:

- `Exchange.setUserPauseBlockInterval`.
- `Exchange.preapproveOrder`.
- `OracleModule.pauseResolver`.
- `PositionManager.setCrossModuleAuth`.

While these redundant emissions do not affect on-chain state or solvency, they degrade the reliability of off-chain systems. Indexers, monitoring tools, and audit trails may record “phantom” transitions, leading to misleading historical data or false alerts. Additionally, in functions like `OracleModule.pauseResolver`, repeated calls can refresh pause timestamps, misrepresenting when a condition was first paused.

Recommendation: Ensure that state-change events are only emitted when an actual transition occurs. This can be achieved by checking the current state before updating it. Apply this pattern across all setters for pausing, preapprovals, invalidations, and authorization flags. For timestamp-based pause mechanisms, preserve the original timestamp on repeated calls unless refreshing it is the intended behavior.

Polymarket: Acknowledged.

Cantina Managed: Acknowledged.

3.2.3 Recomposition can increase combinatorial payouts compared to the sum of rounded fractional payouts

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: Combinatorial NO positions can be decomposed into lower-dimensional claims through `extract()` or `convertToYesBasket()`, and later reconstructed through `inject()` or `mergeFromYesBasket()`. As shown by `test_extractPayoutWeakInequality_fractional` and `test_yesBasketPayoutWeakInequality_fractional` present within `CombinatorialModule.t.sol`, fractional resolution of the underlying legs can result in the rounded collateral payout of the decomposed claims being lower than the rounded payout of the full NO position.

Considering the reverse scenario, in which the decomposed claims are reconstructed to the full NO position, `inject()` can consume two positions whose individual rounded payouts sum to 0 and mint a full NO position redeemable for 1. `mergeFromYesBasket()` can do the same when reconstructing from the canonical YES basket, both scenarios assuming that the fractional payout appears as the first leg in the sorted underlying basket.

While the concern is that this behavior could be recursively leveraged to extract value from the protocol, reversing rounding in favor of the protocol by recomposing full NO positions, it does not by itself appear to represent an overpayment. The exact semantic value of the position set is preserved, since the source positions must first be created by splitting collateral or by decomposing an already-funded position, and the final redeemed collateral does not exceed the collateral used to mint those source positions.

Proof of Concept: The following tests demonstrate this behavior:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.15;

import { ConditionId, ConditionIdLib, EventId, PositionId } from
↳ "@polymarket-v2/src/libraries/Ids.sol";

import { CombinatorialModuleTest } from "./CombinatorialModule.t.sol";

contract RecompositionPoCTest is CombinatorialModuleTest {
    uint256 internal constant UNIT = 1;

    struct FractionalNegRiskLegs {
        uint256 real0Yes;
        uint256 real0No;
        uint256 real1Yes;
        uint256 real1No;
        uint256 otherYes;
        uint256 otherNo;
    }

    struct ReconstructionPositions {
        PositionId prefixYes;
        PositionId reducedNo;
        PositionId residualYes;
        PositionId fullNo;
    }
}
```

```

    uint256 suffixIndex;
}

struct BasketPositions {
    ConditionId firstBasketConditionId;
    ConditionId secondBasketConditionId;
    PositionId firstBasketYes;
    PositionId secondBasketYes;
    PositionId fullNo;
}

function setUp() public override {
    super.setUp();

    vm.prank(admin);
    positions.negRiskModule.addResolver(oracle);
}

function _resolveFractionalNegRiskEvent() internal returns (FractionalNegRiskLegs
↳ memory negRiskLegs) {
    EventId eventId = positions.negRiskModule.getEventId(2,
↳ "neg-risk-recomposition");
    ConditionId real0 = eventId.computeConditionId(0);
    ConditionId real1 = eventId.computeConditionId(1);
    ConditionId other = eventId.computeConditionId(2);

    // A fractional real condition plus an all-NO real condition auto-derives
↳ fractional Other.
    _reportNegRiskResult(real0, RESULT_DENOMINATOR / 2);
    _reportNegRiskResult(real1, 0);

    uint256[] memory otherResult = positions.negRiskModule.getResult(other);
    assertEq(otherResult[0], RESULT_DENOMINATOR / 2);
    assertEq(otherResult[1], RESULT_DENOMINATOR / 2);

    negRiskLegs = FractionalNegRiskLegs({
        real0Yes: PositionId.unwrap(real0.computePositionId(0)),
        real0No: PositionId.unwrap(real0.computePositionId(1)),
        real1Yes: PositionId.unwrap(real1.computePositionId(0)),
        real1No: PositionId.unwrap(real1.computePositionId(1)),
        otherYes: PositionId.unwrap(other.computePositionId(0)),
        otherNo: PositionId.unwrap(other.computePositionId(1))
    });
}

function _reportNegRiskResult(ConditionId conditionId, uint256 yesPayout) internal {
    uint256[] memory result = new uint256[](2);
    result[0] = yesPayout;
    result[1] = RESULT_DENOMINATOR - yesPayout;

    vm.prank(oracle);
    positions.negRiskModule.reportResult(conditionId, result);
}

function _prepareReconstructionPositions(uint256 otherYes) internal returns
↳ (ReconstructionPositions memory ids) {
    uint256[] memory prefixLegs = _sorted2(otherYes, conditionBY);
    uint256[] memory fullLegs = _sorted3(otherYes, conditionBY, conditionCY);
    uint256[] memory residualLegs = _sorted3(otherYes, conditionBY, conditionCN);

    ConditionId prefix = combinatorial.prepareCondition(_positionIds(prefixLegs));
    ConditionId full = combinatorial.prepareCondition(_positionIds(fullLegs));
    ConditionId residual =
↳ combinatorial.prepareCondition(_positionIds(residualLegs));

    ids = ReconstructionPositions({

```

```

        prefixYes: prefix.computePositionId(0),
        reducedNo: prefix.computePositionId(1),
        residualYes: residual.computePositionId(0),
        fullNo: full.computePositionId(1),
        suffixIndex: _indexOf(fullLegs, conditionCY)
    });
}

function _prepareFractionalPrefixBasketPositions(FractionalNegRiskLegs memory
    ↪ negRiskLegs)
    internal
    returns (BasketPositions memory ids)
{
    // Both legs are from the same neg-risk event, so condition index 0 sorts before
    ↪ index 1.
    // This makes the fractional real0 leg the basket prefix and the false real1 leg
    ↪ the suffix:
    // NO(real0^real1) basket = YES(!real0) + YES(real0^!real1).
    ids = _prepareBasketPositions({
        fullLegs: _sorted2(negRiskLegs.real0Yes, negRiskLegs.real1Yes),
        firstBasketLegs: _singleCondition(negRiskLegs.real0No),
        secondBasketLegs: _sorted2(negRiskLegs.real0Yes, negRiskLegs.real1No)
    });
}

function _prepareFullValueFirstBasketPositions(FractionalNegRiskLegs memory
    ↪ negRiskLegs)
    internal
    returns (BasketPositions memory ids)
{
    // Binary module ids sort before neg-risk module ids. With Q = C^0Other and C
    ↪ false,
    // the canonical basket starts with YES(!C), which is already full value:
    // NO(C^0Other) basket = YES(!C) + YES(C^!0Other).
    ids = _prepareBasketPositions({
        fullLegs: _sorted2(conditionCY, negRiskLegs.otherYes),
        firstBasketLegs: _singleCondition(conditionCN),
        secondBasketLegs: _sorted2(conditionCY, negRiskLegs.otherNo)
    });
}

function _prepareBasketPositions(
    uint256[] memory fullLegs,
    uint256[] memory firstBasketLegs,
    uint256[] memory secondBasketLegs
) internal returns (BasketPositions memory ids) {
    ConditionId fullConditionId =
    ↪ combinatorial.prepareCondition(_positionIds(fullLegs));
    ConditionId firstBasketConditionId =
    ↪ combinatorial.prepareCondition(_positionIds(firstBasketLegs));
    ConditionId secondBasketConditionId =
    ↪ combinatorial.prepareCondition(_positionIds(secondBasketLegs));

    ids = BasketPositions({
        firstBasketConditionId: firstBasketConditionId,
        secondBasketConditionId: secondBasketConditionId,
        firstBasketYes: firstBasketConditionId.computePositionId(0),
        secondBasketYes: secondBasketConditionId.computePositionId(0),
        fullNo: fullConditionId.computePositionId(1)
    });
}

function _alicePair() internal view returns (address[] memory to) {
    to = new address[](2);
    to[0] = alice;
    to[1] = alice;
}

```

```

}

function _splitToAlice(address[] memory to, ConditionId conditionId, uint256 amount)
↳ internal {
    collateral.token.transfer(address(combinatorial), amount);
    combinatorial.split(to, conditionId, amount);
}

function _sourceBasketPositions(BasketPositions memory ids, address[] memory to,
↳ uint256 amount) internal {
    _splitToAlice(to, ids.firstBasketConditionId, amount);
    _splitToAlice(to, ids.secondBasketConditionId, amount);
}

function _transferToCombinatorial(PositionId positionId, uint256 amount) internal {
    positions.manager.safeTransferFrom(alice, address(combinatorial),
↳ PositionId.unwrap(positionId), amount, "");
}

function _redeemFromCombinatorial(PositionId positionId, uint256 amount) internal {
    _transferToCombinatorial(positionId, amount);
    combinatorial.redeem(alice, positionId, amount);
}

function test_reconstructNoOverpaysFromFractionalPrefixSources() public {
    uint256 amount = UNIT;

    FractionalNegRiskLegs memory negRiskLegs = _resolveFractionalNegRiskEvent();

    // Binary legs used as the resolved true/dead suffixes in the prefix, residual,
    ↳ and full NO.
    _resolve(condB, true);
    _resolve(condC, false);

    ReconstructionPositions memory ids =
    ↳ _prepareReconstructionPositions(negRiskLegs.otherYes);

    uint256 sourceValue =
        combinatorial.getPayout(ids.reducedNo, amount) +
        ↳ combinatorial.getPayout(ids.residualYes, amount);
    assertEq(sourceValue, 0);
    assertEq(combinatorial.getPayout(ids.fullNo, amount), amount);

    _dealCollateral(alice, amount);

    vm.startPrank(alice);
    positions.manager.setApprovalForAll(address(combinatorial), true);

    address[] memory to = _alicePair();
    _splitToAlice(to, ids.prefixYes.conditionId(), amount);
    _transferToCombinatorial(ids.prefixYes, amount);
    combinatorial.splitOnCondition(to, ids.prefixYes, ConditionIdLib.from(condC),
    ↳ amount);

    // Two zero-payout source claims are accepted and become a full-value NO.
    _transferToCombinatorial(ids.reducedNo, amount);
    _transferToCombinatorial(ids.residualYes, amount);
    combinatorial.inject(alice, ids.fullNo, ids.suffixIndex, amount);

    assertEq(positions.manager.balanceOf(alice, PositionId.unwrap(ids.reducedNo)),
    ↳ 0);
    assertEq(positions.manager.balanceOf(alice, PositionId.unwrap(ids.residualYes)),
    ↳ 0);
    assertEq(positions.manager.balanceOf(alice, PositionId.unwrap(ids.fullNo)),
    ↳ amount);
}

```

```

_redeemFromCombinatorial(ids.fullNo, amount);
vm.stopPrank();

assertEq(collateral.token.balanceOf(alice), amount);
}

function test_mergeFromYesBasketOverpaysFromFractionalBasket() public {
    uint256 amount = UNIT;

    FractionalNegRiskLegs memory negRiskLegs = _resolveFractionalNegRiskEvent();
    BasketPositions memory ids =
        ↪ _prepareFractionalPrefixBasketPositions(negRiskLegs);

    // The false leg makes NO(real0^real1) full value, but the basket reaches that
    ↪ false leg
    // only after multiplying by the fractional real0 prefix. Both basket terms round
    ↪ to zero.
    uint256 basketValue =
        combinatorial.getPayout(ids.firstBasketYes, amount) +
        ↪ combinatorial.getPayout(ids.secondBasketYes, amount);
    assertEq(basketValue, 0);
    assertEq(combinatorial.getPayout(ids.fullNo, amount), amount);

    _dealCollateral(alice, 2 * amount);

    vm.startPrank(alice);
    positions.manager.setApprovalForAll(address(combinatorial), true);

    _sourceBasketPositions(ids, _alicePair(), amount);

    _transferToCombinatorial(ids.firstBasketYes, amount);
    _transferToCombinatorial(ids.secondBasketYes, amount);
    combinatorial.mergeFromYesBasket(alice, ids.fullNo, amount);

    assertEq(positions.manager.balanceOf(alice,
        ↪ PositionId.unwrap(ids.firstBasketYes)), 0);
    assertEq(positions.manager.balanceOf(alice,
        ↪ PositionId.unwrap(ids.secondBasketYes)), 0);
    assertEq(positions.manager.balanceOf(alice, PositionId.unwrap(ids.fullNo)),
        ↪ amount);

    _redeemFromCombinatorial(ids.fullNo, amount);
    vm.stopPrank();

    assertEq(collateral.token.balanceOf(alice), amount);
}

function test_mergeFromYesBasketNoDiscrepancyWhenFullValueTermSortsFirst() public {
    uint256 amount = UNIT;

    FractionalNegRiskLegs memory negRiskLegs = _resolveFractionalNegRiskEvent();
    _resolve(condC, false);

    BasketPositions memory ids = _prepareFullValueFirstBasketPositions(negRiskLegs);

    uint256 firstBasketPayout = combinatorial.getPayout(ids.firstBasketYes, amount);
    uint256 secondBasketPayout = combinatorial.getPayout(ids.secondBasketYes,
        ↪ amount);
    uint256 fullNoPayout = combinatorial.getPayout(ids.fullNo, amount);

    // Here the false leg sorts first, so the basket contains YES(!false) as its
    ↪ first term.
    // The burned basket is already worth the same amount as the minted full NO.
    assertEq(firstBasketPayout, amount);
    assertEq(secondBasketPayout, 0);
    assertEq(firstBasketPayout + secondBasketPayout, fullNoPayout);
}

```



```

    _dealCollateral(alice, 2 * amount);

    vm.startPrank(alice);
    positions.manager.setApprovalForAll(address(combinatorial), true);

    _sourceBasketPositions(ids, _alicePair(), amount);

    _transferToCombinatorial(ids.firstBasketYes, amount);
    _transferToCombinatorial(ids.secondBasketYes, amount);
    combinatorial.mergeFromYesBasket(alice, ids.fullNo, amount);

    _redeemFromCombinatorial(ids.fullNo, amount);
    vm.stopPrank();

    assertEq(collateral.token.balanceOf(alice), amount);
}
}

```

Recommendation: Consider explicitly documenting this path-dependence in the combinatorial module specification and the intended property that decomposition/recomposition preserve exact value, while rounded basket payouts can be less than the full `NO` payout.

Polymarket: Acknowledged.

Cantina Managed: Acknowledged.

3.2.4 Potential ABI Mismatch Between `CollateralToken` and Ramps

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The removal of callbacks from `CollateralToken` introduced a breaking ABI change by replacing the five-argument `wrap` and `unwrap` functions with three-argument versions. If the `CollateralToken` proxy is upgraded to the new implementation while the existing ramp contracts (`CollateralOnramp`, `CollateralOfframp`, and `PermissionedRamp`) remain on the previous bytecode generation, a selector mismatch will occur. The ramps will continue to call the legacy five-argument selectors, which are no longer exposed by the new implementation. This incompatibility will cause all collateral wrapping and unwrapping operations to revert, disabling pUSD ingress and egress.

Recommendation: Upgrade `CollateralToken` and all associated ramp contracts together as a matched deployment unit to ensure ABI compatibility. Alternatively, implement five-argument compatibility overloads in the new `CollateralToken` that forward to the three-argument logic while rejecting non-empty callback parameters.

Polymarket: Acknowledged, all upgrades will be done together.

Cantina Managed: Acknowledged.